

3. async await

ES2017에 추가된 async await는 프로미스의 then 메서드 체인 형식보다 가독성이 향상됩니다. 그러나 async await가 완전히 프로미스의 기능을 대체할 수 없습니다.

- ECMAScript 2017 JavaScript 에디션의 일부인 async functions 그리고 await 키워드는 ECMAScript2017에 추가되었습니다. 이 기능들은 기본적으로 비동기 코드를 쓰고 Promise를 더 읽기 더 쉽도록 합니다.
- 비동기 함수를 async 함수로 만들기 위하여 function()앞에 async 키워드를 추가합니다.
- async function()은 await 키워드가 비동기 코드를 호출할 수 있게 해주는 함수
- fulfil Promise가 반환되기 때문에 반환된 값을 사용하기 위해선 .then() 블록 사용
- await 키워드는 웹 API를 포함하여 Promise를 반환하는 함수를 호출할 때 사용
- await는 async function 안에서만 쓸 수 있음

1. 기본 예제

```
{% code lineNumbers="true" %}
```

```
function helloSync() { return "Hello" };
console.log(helloSync());    // Hello

async function helloAsync() { return "Hello" };
console.log(helloAsync());    // Promise {<fulfilled>: 'Hello'}
```



```
helloAsync().then((value=>console.log(value))); // Hello
```

```
{% endcode %}
```

- 1~2 line : helloSync 함수는 일반 함수 이므로 2 line에서 실행되어 "Hello"를 표시 합니다.
- 4~5 line : helloAsync 함수는 async를 붙여서 Promise 로 선언한 함수로 5 line 에서 Promise 상태를 표시 합니다.
- 7 line : then을 만나면 최종 결과를 받아 처리 합니다. ("Hello" 출력)
 - fulfil Promise가 반환되기 때문에 반환된 값을 사용하기 위해선 .then() 블록 사용

```
async function hello01() {
  return greeting = await Promise.resolve("Hello 01");
};

console.log("start.....");
hello01().then(value => console.log(value));
console.log( hello01() );
console.log("end.....");
```

```

start.....
▼ Promise {<pending>} i
  ► [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: "Hello 01"
end.....
Hello 01

```

2. err 처리

```
{% code lineNumbers="true" %}
```

```

async function asyncError(isParam) {
  try {
    if (isParam) {
      return await Promise.resolve("Hello 01");
    } else {
      throw new Error("ERROR");
    }
  } catch(e) {
    console.log("E ", e); // Stacktrace
    return e.message;
  }
};

asyncError(true).then((value) => {
  console.log("param true : " + value);
}).catch((value)=>{
  console.log("param true err: " + value);
});

asyncError(false).then((value) => {
  console.log("param false : " + value);
}).catch((value)=>{
  console.log("param false err: " + value);
});

```

```
{% endcode %}
```

- 14 line : asyncError async 함수에 true로 파라미터 넘김
 - 4 line : 프로미스 실행
- 20 line : asyncError async 함수에 false 로 파라미터 넘김
 - 6 line : catch 블록에서 에러 메세지 출력 후
 - 22 line : catch 로 받아서 23 line 에러 메세지 출력
- 14 line : then으로 성공 결과 받아서
 - 15 line : 성공 메세지 출력

```
E Error: ERROR
  at asyncError (<anonymous>:6:19)
  at <anonymous>:20:1

parem false : ERROR

parem true : Hello 01
```

3. 비동기 순차 처리

```
// 기명 함수
const sum = function sum (num1, num2) {
  console.log("sum : pending" );
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      let result = num1 + num2;
      result = `${num1} + ${num2} = ${result}`;
      resolve(result);
    }, 2000);
  });
}

// 익명 함수
const minus = function (num1, num2) {
  console.log("minus : pending" );
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      let result = num1 - num2;
      result = `${num1} - ${num2} = ${result}`;
      resolve(result);
    }, 1500);
  });
}

// 화살표 함수 적용
const multiply = (num1, num2) => {
  console.log("multiply : pending" );
  return new Promise((resolve, reject) => {
    setTimeout( () => {
      let result = num1 * num2;
      result = `${num1} * ${num2} = ${result}`;
      resolve(result);
    }, 1000);
  });
}

// 화살표 함수 적용
const divide = (num1, num2) => {
  console.log("divide : pending" );
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let result = num1 / num2;
      result = `${num1} / ${num2} = ${result}`;
    }, 1000);
  });
}
```

```

        resolve(result);
    }, 1000);
});
}

```

3-1. 비동기 순차 처리

```

async function fnAsync() {
    const res1 = await sum(100, 200);
    console.log(res1);
    const res2 = await minus(100, 200);
    console.log(res2);
    const res3 = await multiply(100, 200);
    console.log(res3);
    const res4 = await divide(100, 200);
    console.log(res4);
}

fnAsync();

```

```

sum : pending
: ▶ Promise {<pending>}
100 + 200 = 300
minus : pending
100 - 200 = -100
multiply : pending
100 * 200 = 20000
divide : pending
100 / 200 = 0.5

```

3-2. 비동기 병렬 처리

```

async function fnParallelAsync() {
    const p1 = sum(100, 200);
    const p2 = minus(100, 200);
    const p3 = multiply(100, 200);
    const p4 = divide(100, 200);

    const res1 = await p1;
    const res2 = await p2;
    const res3 = await p3;
    const res4 = await p4;

    console.log(res1);
}

```

```

    console.log(res2);
    console.log(res3);
    console.log(res4);
  }

  fnParallelAsync();

```

```

sum    : pending
minus  : pending
multiply : pending
divide : pending
❯ ▶ Promise {<pending>}
100 + 200 = 300
100 - 200 = -100
100 * 200 = 20000
100 / 200 = 0.5

```

3-3. Promise.all 사용

```

async function fnPromiseAll() {
  const [res1, res2, res3, res4] = await Promise.all([
    sum(100, 200),
    minus(100, 200),
    multiply(100, 200),
    divide(100, 200)
  ]);

  console.log(res1);
  console.log(res2);
  console.log(res3);
  console.log(res4);
}

fnParallelAsync();

```

```

sum : pending
minus : pending
multiply : pending
divide : pending
❯ ▶ Promise {<pending>}
100 + 200 = 300
100 - 200 = -100
100 * 200 = 20000
100 / 200 = 0.5

```

3-4. Promise.race 사용

```

async function fnPromiseRace() {
  const reponse = await Promise.race([
    sum(100, 200),
    minus(100, 200),
    multiply(100, 200),
    divide(100, 200)
  ]);

  console.log(reponse);
}

fnPromiseRace();

```

```

sum : pending
minus : pending
multiply : pending
divide : pending
❯ ▶ Promise {<pending>}
100 * 200 = 20000

```